
Demostración Práctica

Pipeline de Integración Continua para Software Embebido

Docker + GitHub + Jenkins

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Taller de Sistemas Embebidos

Prof. Ing. Johan Carvajal Godínez

Daniel Chavarría García

18 de septiembre de 2026

Alcance de este documento

Registro completo y verificado de la demostración práctica del mini-taller. Cada paso incluye el comando exacto ejecutado, la salida obtenida y un punto de verificación. Los problemas documentados en la sección final son fallos reales encontrados durante la construcción, no escenarios hipotéticos.

El caso de estudio es una máquina de estados de control de acceso escrita en C, elegida porque su comportamiento se explica en una frase y porque un cambio de un solo carácter basta para provocar un fallo observable.

Repositorio: <https://github.com/Daniell03/Taller-Jenkins>

Índice

1. Objetivo y arquitectura de la demostración	3
2. Requisitos previos	4
3. Fase 1 — Código fuente	4
3.1. Interfaz del módulo	4
3.2. Implementación de la máquina de estados	4
4. Fase 2 — Pruebas unitarias	6
4.1. Las seis pruebas	6
4.2. Prueba clave del umbral	6
5. Fase 3 — El Makefile	6
6. Fase 4 — Infraestructura Docker	7
6.1. Dockerfile	7
6.2. docker-compose.yml	7
6.3. Construcción y arranque	8
6.4. Clave inicial y asistente	8
7. Fase 5 — El pipeline	8
7.1. Jenkinsfile comentado	8
7.2. Las siete directivas y lo que enseñan	10
7.3. Configuración del trabajo en Jenkins	10
8. Fase 6 — Ejecución y evidencias	11
8.1. Ejecución #1: Ejecución correcta	11
8.2. Ejecución #2: el fallo provocado	11
8.3. Ejecución #3: confirmación	11
8.4. Ejecución #4: disparo automático	12
9. Problemas encontrados y su solución	12
10. Qué queda demostrado	13

1. Objetivo y arquitectura de la demostración

La demostración evidencia el ciclo completo de integración continua aplicado a software embebido, articulando tres herramientas:

Herramienta	Rol	Qué demuestra
Docker	Aloja el controlador de Jenkins con el entorno de compilación incluido	La imagen oficial no trae compilador: el entorno de construcción debe empaquetarse
GitHub	Repositorio remoto del código y del pipeline	<i>Pipeline as Code</i> : el proceso se versiona junto al código
Jenkins	Orquesta la ejecución de las etapas	Automatización de construir, probar y entregar, con política de calidad aplicada sin intervención humana

El caso de estudio

Una máquina de estados finitos que modela una puerta con control de acceso. Cuatro estados y cuatro eventos:

Estado	Significado
CERRADO	Reposo, puerta asegurada
AUTORIZANDO	Credencial recibida, validando
ABIERTO	Acceso concedido
BLOQUEADO	Estado terminal tras tres fallos consecutivos

El módulo es puro: no contiene hardware ni entrada/salida, de modo que sus pruebas pueden ejecutarse dentro del contenedor de Jenkins sin necesidad de una tarjeta física.

Por qué este ejemplo. El umbral de bloqueo está definido como una constante. Cambiar ese valor de 3 a 2 produce código que *compila sin errores* pero cuyo comportamiento ya no es el especificado. Esa distinción entre “no compila” y “compila pero está mal” es el eje de la demostración.

Estructura del repositorio

```
Taller-Jenkins/
|- src/                <- código fuente
|  |- puerta.h          (interfaz y constantes)
|  |- puerta.c          (implementación de la FSM)
|  |- main.c            (ejecutable de demostración)
|- tests/              <- pruebas
|  |- minitest.h        (micro marco que emite JUnit XML)
|  |- test_puerta.c     (6 pruebas unitarias)
|- docker-demo/        <- infraestructura
|  |- Dockerfile         (Jenkins + entorno de compilación)
|  |- docker-compose.yml (servicio y volumen persistente)
|  |- plugins.txt        (plugins precargados)
|- Makefile             <- objetivos de construcción
|- Jenkinsfile          <- definición del pipeline
```

Esta organización refleja las cuatro etapas del proceso: código fuente (`src/`), pruebas (`tests/`), integración (`Jenkinsfile` y `Makefile`) y entrega (`build/dist/`, generado y no versionado).

2. Requisitos previos

Componente	Versión usada	Verificación
WSL2 con Ubuntu	24.04 LTS	<code>lsb_release -a</code>
Docker Engine	29.8.0	<code>docker --version</code>
Docker Compose	v2	<code>docker compose version</code>
Git	2.43.0	<code>git --version</code>
Cuenta de GitHub	—	Token de acceso personal con alcance <code>repo</code>

Sobre el token de GitHub. Desde 2021 GitHub no acepta contraseñas para operaciones de Git. Se requiere un token de acceso personal, y debe tener el alcance `repo` habilitado. Un token *fine-grained* sin permiso de escritura sobre el repositorio produce un error 403 aunque la autenticación sea correcta.

El token es equivalente a una contraseña: no debe escribirse en archivos del repositorio ni en documentación. Si se expone accidentalmente, debe revocarse de inmediato, ya que borrarlo de un archivo no lo elimina del historial de Git.

3. Fase 1 — Código fuente

3.1 Interfaz del módulo

```

1 #ifndef PUERTA_H
2 #define PUERTA_H
3
4 #include <stdint.h>
5
6 #define MAX_INTENTOS 3u    /* bloqueo tras este numero de fallos */
7
8 typedef enum {
9     EST_CERRADO = 0,
10    EST_AUTORIZANDO,
11    EST_ABIERTO,
12    EST_BLOQUEADO
13 } estado_t;
14
15 typedef enum {
16     EV_CREDENCIAL = 0,
17     EV_OK,
18     EV_FALLO,
19     EV_TIEMPO
20 } evento_t;
21
22 typedef struct {
23     estado_t estado;
24     uint8_t intentos;
25 } puerta_t;
26
27 void      puerta_init(puerta_t *p);
28 estado_t  puerta_evento(puerta_t *p, evento_t ev);
29 const char *puerta_nombre(estado_t e);
30
31 #endif /* PUERTA_H */

```

3.2 Implementación de la máquina de estados

```

1 #include "puerta.h"
2 #include <stddef.h>
3
4 void puerta_init(puerta_t *p)
5 {
6     if (p == NULL) {
7         return;
8     }
9     p->estado = EST_CERRADO;
10    p->intentos = 0u;
11 }
12
13 estado_t puerta_evento(puerta_t *p, evento_t ev)
14 {
15     if (p == NULL) {
16         return EST_BLOQUEADO; /* falla segura */
17     }
18
19     switch (p->estado) {
20
21     case EST_CERRADO:
22         if (ev == EV_CREDENCIAL) {
23             p->estado = EST_AUTORIZANDO;
24         }
25         break;
26
27     case EST_AUTORIZANDO:
28         if (ev == EV_OK) {
29             p->estado = EST_ABIERTO;
30             p->intentos = 0u; /* éxito: reinicia contador */
31         } else if (ev == EV_FALLO) {
32             p->intentos++;
33             p->estado = (p->intentos >= MAX_INTENTOS)
34                 ? EST_BLOQUEADO
35                 : EST_CERRADO;
36         }
37         break;
38
39     case EST_ABIERTO:
40         if (ev == EV_TIEMPO) {
41             p->estado = EST_CERRADO; /* cierre automatico */
42         }
43         break;
44
45     case EST_BLOQUEADO:
46     default:
47         /* Estado terminal: solo se sale reiniciando. */
48         break;
49     }
50
51     return p->estado;
52 }

```

Dos decisiones de diseño con intención didáctica. Un puntero nulo devuelve BLOQUEADO en lugar de abrir la puerta: falla segura. Y un acceso exitoso reinicia el contador de intentos, de modo que tres fallos separados por un acierto no bloquean el sistema.

✓ **Punto de verificación.** El módulo debe compilar sin advertencias bajo `-Wall -Wextra -Werror`:

```
gcc -std=c11 -Wall -Wextra -Werror -Isrc -c src/puerta.c -o /tmp/p.o
```

El uso de `-Werror` convierte cualquier advertencia en error. Es la misma política que aplicará el pipeline.

4. Fase 2 — Pruebas unitarias

Las pruebas se apoyan en un micro marco propio que emite un informe en formato JUnit XML. Ese formato es el que Jenkins lee para poblar la pestaña de resultados, mostrando cada prueba por nombre en lugar de obligar a leer el registro completo.

4.1 Las seis pruebas

Prueba	Qué verifica
test_init_arranca_cerrado	Estado inicial y contador en cero
test_credencial_pasa_a_autorizando	Transición válida desde reposo
test_autorizacion_correcta_abre	Camino feliz completo
test_cierre_por_tiempo	Cierre automático tras apertura
test_bloqueo_tras_tres_fallos	Umbral de bloqueo (esta es la que se romperá)
test_bloqueado_es_terminal	Ningún evento saca del estado terminal

4.2 Prueba clave del umbral

```

1 static void test_bloqueo_tras_tres_fallos(void)
2 {
3     puerta_t p;
4     unsigned i;
5     puerta_init(&p);
6
7     /* Dos fallos: vuelve a CERRADO, aun sin bloquear. */
8     for (i = 0u; i < 2u; ++i) {
9         puerta_evento(&p, EV_CREDENCIAL);
10        MT_ASSERT_EQ(puerta_evento(&p, EV_FALLO), EST_CERRADO);
11    }
12
13    /* Tercer fallo: bloqueo. */
14    puerta_evento(&p, EV_CREDENCIAL);
15    MT_ASSERT_EQ(puerta_evento(&p, EV_FALLO), EST_BLOQUEADO);
16 }

```

✓ **Punto de verificación.** Ejecutar las pruebas localmente:

```

$ make test
[ ok ] test_init_arranca_cerrado
[ ok ] test_credencial_pasa_a_autorizando
[ ok ] test_autorizacion_correcta_abre
[ ok ] test_cierre_por_tiempo
[ ok ] test_bloqueo_tras_tres_fallos
[ ok ] test_bloqueado_es_terminal

minitest: informe JUnit en build/reports/puerta.xml
Resumen: 6 pruebas, 0 fallas

```

5. Fase 3 — El Makefile

```

APP      := control-acceso
VERSION  ?= 0.1.0

```

```
CC      ?= gcc
CFLAGS := -std=c11 -Wall -Wextra -Werror -O2 -Isrc \
          -DAPP_VERSION="\$(VERSION)\\"

all:      # compila el ejecutable
test:     # compila y ejecuta pruebas; genera JUnit XML
package:  # empaqueta el binario en un .tar.gz versionado
clean:    # borra todo lo generado
help:     # lista los objetivos disponibles
```

Regla de oro. El pipeline no contiene lógica de compilación: únicamente invoca objetivos de este Makefile. La consecuencia práctica es que cualquier participante puede ejecutar exactamente lo mismo en su máquina sin instalar Jenkins, y que un fallo del build se reproduce localmente sin depender del servidor.

La variable `VERSION` usa `?` para que el pipeline pueda sobrescribirla con el número de ejecución, ligando cada artefacto al build que lo produjo.

6. Fase 4 — Infraestructura Docker

6.1 Dockerfile

```
# La imagen oficial de Jenkins trae Java y el servidor, pero NO trae
# compilador. Si el pipeline va a ejecutar "make", el entorno de
# construccion tiene que existir en la imagen.

FROM jenkins/jenkins:lts-jdk21

USER root
RUN apt-get update \
    && apt-get install -y --no-install-recommends \
        build-essential make file cppcheck \
    && rm -rf /var/lib/apt/lists/*

ENV JAVA_OPTS="-Duser.timezone=America/Costa_Rica"

COPY plugins.txt /usr/share/jenkins/ref/plugins.txt
RUN jenkins-plugin-cli --plugin-file /usr/share/jenkins/ref/plugins.txt

USER jenkins
```

6.2 docker-compose.yml

```
services:
  jenkins:
    build:
      context: .
      dockerfile: Dockerfile
    image: jenkins-taller:v1
    container_name: jenkins-taller
    restart: unless-stopped
    ports:
      - "8080:8080"          # interfaz web
      - "50000:50000"        # comunicacion con agentes
    volumes:
      - jenkins_home:/var/jenkins_home

volumes:
  jenkins_home:
```

Volumen nombrado, no *bind mount*. La documentación oficial desaconseja montar una carpeta del sistema anfitrión sobre `/var/jenkins_home`, porque el usuario `jenkins` dentro del contenedor opera con un identificador específico y esto produce conflictos de permisos. El directorio debe tratarse con el mismo cuidado que la base de datos de un sistema.

6.3 Construcción y arranque

```
$ cd docker-demo
$ DOCKER_BUILDKIT=0 docker build -t jenkins-taller:v1 .
$ docker compose up -d --no-build
$ docker compose ps
```

NAME	IMAGE	STATUS	PORTS
jenkins-taller	jenkins-taller:v1	Up 2 seconds	0.0.0.0:8080->8080/tcp

✓ **Punto de verificación.** Confirmar que el entorno de compilación quedó dentro de la imagen:

```
$ docker exec jenkins-taller bash -c \
    "gcc --version | head -1; make --version | head -1"
gcc (Debian 14.2.0-19) 14.2.0
GNU Make 4.4.1
```

Esta verificación es la evidencia de que el pipeline podrá ejecutar `make` sin instalar nada en tiempo de ejecución.

6.4 Clave inicial y asistente

```
$ docker exec jenkins-taller \
    cat /var/jenkins_home/secrets/initialAdminPassword
```

Con esa clave se accede a `http://localhost:8080`. En la pantalla de complementos se elige *Select plugins to install* y luego *None*, ya que los necesarios vienen precargados desde el `Dockerfile`. Esto elimina la dependencia de la red durante la puesta en marcha.

7. Fase 5 — El pipeline

7.1 Jenkinsfile comentado

```
pipeline {                                     // raíz de la sintaxis declarativa

    // AGENTE: donde se ejecuta el trabajo. "any" = cualquier nodo.
    // En produccion esto es mala practica: el build pesado debe ir
    // a un agente dedicado, no al controlador.
    agent any

    // OPCIONES: politicas operativas del pipeline completo.
    options {
        timestamps()                          // hora en cada linea
        timeout(time: 10, unit: "MINUTES")   // aborta build colgado
        buildDiscarder(logRotator(numToKeepStr: "10"))
        disableConcurrentBuilds()
    }

    // DISPARADORES: cierran el ciclo de integracion continua.
    // pollSCM consulta el repositorio cada 2 minutos y lanza una
    // ejecucion SOLO si detecta commits nuevos.
    //
    // La "H" reparte la carga: con "*/2" todos los trabajos
    // consultarian en el mismo instante; con "H" cada uno recibe
```

```

// un desfase distinto.
//
// En produccion se usaria un webhook: GitHub avisa a Jenkins en
// el momento del push, sin consultas periodicas. Aqui no es
// viable porque el servidor corre en localhost y GitHub no
// puede alcanzarlo sin un tunel.
triggers {
    pollSCM("H/2 * * * *")
}

// ENTORNO: BUILD_NUMBER lo inyecta Jenkins. El artefacto queda
// ligado a la ejecucion exacta que lo produjo: trazabilidad.
environment {
    VERSION = "0.1.${BUILD_NUMBER}"
}

stages {

    // ETAPA 1: deja constancia de que compilador se uso.
    stage("Preparar") {
        steps {
            echo "Construyendo version ${VERSION}"
            sh "make clean"
            sh "gcc --version | head -1"
        }
    }

    // ETAPA 2 (SOURCE CODE): delega en el Makefile.
    // Si falla, el build queda ROJO y nada mas se ejecuta.
    stage("Compilar") {
        steps {
            sh "make all VERSION=${VERSION}"
        }
    }

    // ETAPA 3 (TESTING): catchError cambia el resultado a
    // UNSTABLE (AMARILLO) en vez de FAILURE (ROJO).
    // ROJO = no compilo, el codigo esta roto
    // AMARILLO = compilo, pero el comportamiento no es el
    // esperado
    stage("Probar") {
        steps {
            catchError(buildResult: "UNSTABLE",
                        stageResult: "UNSTABLE") {
                sh "make test VERSION=${VERSION}"
            }
        }
        post {
            always {
                // Lee el JUnit XML y lo muestra en la pestana
                // "Test Result": cada prueba por nombre.
                junit allowEmptyResults: true,
                    testResults: "build/reports/*.xml"
            }
        }
    }

    // ETAPA 4 (INTEGRACION): el "when" es la politica de calidad
    // automatizada. Si las pruebas dejaron el build en UNSTABLE,
    // esta etapa se OMITE.
    stage("Empaquetar") {
        when {
            expression { currentBuild.result == null ||
                        currentBuild.result == "SUCCESS" }
        }
        steps {
            sh "make package VERSION=${VERSION}"
        }
    }
}

```

```

    }

    // ETAPA 5 (ENTREGA): archiveArtifacts copia el .tar.gz hacia
    // el CONTROLADOR. Por eso sobrevive al cleanWs posterior.
    stage("Entregar") {
        when {
            expression { currentBuild.result == null ||
                          currentBuild.result == "SUCCESS" }
        }
        steps {
            archiveArtifacts artifacts: "build/dist/*.tar.gz",
                             fingerprint: true
        }
    }
}

// POST GLOBAL: acciones segun el resultado final.
post {
    success { echo "SUCCESS: artefacto ${VERSION} listo" }
    unstable { echo "UNSTABLE: compilo, pero fallaron pruebas" }
    failure { echo "FAILURE: revise la consola" }
    always { cleanWs() }
}
}

```

El disparador no se activa solo. Jenkins debe leer el **Jenkinsfile** al menos una vez para enterarse de que existe la directiva **triggers**. Por eso, tras publicar el cambio es necesaria **una** ejecución manual; a partir de ahí el sondeo opera sin intervención.

7.2 Las siete directivas y lo que enseñan

Directiva	Concepto que evidencia
agent	Dónde se ejecuta el trabajo; permite señalar la mala práctica de compilar en el controlador
options	Políticas operativas: tiempo límite, rotación de registros, exclusión mutua
triggers	Cierre del ciclo: la construcción se lanza sin intervención humana
environment	Trazabilidad del artefacto mediante el número de ejecución
catchError	Distinción entre fallo de compilación y fallo de comportamiento
when	Política de calidad aplicada automáticamente, sin decisión humana
post	Acciones condicionadas al resultado; punto natural de notificación

7.3 Configuración del trabajo en Jenkins

Campo	Valor
Nombre	control-acceso-ci
Tipo	Pipeline
Definition	<i>Pipeline script from SCM</i>
SCM	Git
Repository URL	https://github.com/Daniell103/Taller-Jenkins.git
Branch Specifier	*/main
Script Path	Jenkinsfile

La elección de *Pipeline script from SCM* en lugar de pegar el script en el formulario web es lo que materializa el concepto de *Pipeline as Code*: cualquier modificación del proceso de construcción llega por un *commit*, queda revisada y es reversible.

8. Fase 6 — Ejecución y evidencias

8.1 Ejecución #1: Ejecución correcta

```
23:04:36 Construyendo version 0.1.1
23:04:37 + make clean
23:04:38 OK: binario en build/bin/control-acceso (version 0.1.1)
23:04:40 [ ok ] test_init_arranca_cerrado
23:04:40 [ ok ] test_bloqueo_tras_tres_fallos
23:04:40 Resumen: 6 pruebas, 0 fallas
23:04:40 OK: artefacto en build/dist/control-acceso-0.1.1.tar.gz
23:04:41 Archiving artifacts
23:04:42 SUCCESS: artefacto 0.1.1 listo para entrega
Finished: SUCCESS
```

✓ Cuatro evidencias a mostrar en pantalla:

1. Indicador azul o verde en el historial de ejecuciones.
2. *Stage View* con las cinco etapas en verde.
3. Pestaña *Test Result* con las seis pruebas por nombre.
4. *Build Artifacts* con el paquete descargable.

8.2 Ejecución #2: el fallo provocado

El cambio consiste en modificar un único carácter en `src/puerta.h`:

```
-#define MAX_INTENTOS 3u
+#define MAX_INTENTOS 2u

23:25:39 OK: binario en build/bin/control-acceso (version 0.1.2)
23:25:40 [ ok ] test_cierre_por_tiempo
23:25:40 [ FALLA ] test_bloqueo_tras_tres_fallos
23:25:40 Resumen: 6 pruebas, 1 fallas
23:25:40 Setting overall build result to UNSTABLE
23:25:40 Stage "Empaquetar" skipped due to when conditional
23:25:40 Stage "Entregar" skipped due to when conditional
23:25:42 UNSTABLE: compiló, pero fallaron pruebas
Finished: UNSTABLE
```

Este es el momento central de la demostración. La secuencia del registro cuenta la historia completa: el código *compiló sin errores*, pero una prueba detectó que el comportamiento ya no era el especificado. El resultado fue amarillo y no rojo, y las etapas de empaquetado y entrega **se omitieron automáticamente**. Nadie tomó esa decisión: la tomó la condición **when**. El artefacto 0.1.2 nunca llegó a existir, y el último paquete disponible siguió siendo el 0.1.1 de la ejecución anterior.

8.3 Ejecución #3: confirmación

Tras revertir el cambio, la ejecución #3 vuelve a verde y produce `control-acceso-0.1.3.tar.gz`. El ciclo completo queda demostrado: verde, amarillo, verde.

Un detalle que suele confundir. El registro muestra que `cleanWs()` borra el espacio de trabajo *después* de archivar. El artefacto sobrevive porque `archiveArtifacts` ya lo había transferido al controlador. Esto ilustra directamente la separación entre el almacenamiento temporal del agente y la persistencia en el controlador.

8.4 Ejecución #4: disparo automático

Con la directiva `triggers` en su lugar, la publicación de un cambio basta para lanzar una ejecución:

```
$ printf '\n' >> README.md
$ git add README.md
$ git commit -m "Prueba del disparo automatico"
$ git push
```

Transcurridos menos de dos minutos, Jenkins detecta el cambio y construye sin que nadie pulse ningún botón:

```
Started by an SCM change
Obtained Jenkinsfile from git https://github.com/Daniell103/Taller-Jenkins.git
...
Finished: SUCCESS
```

✓ **Punto de verificación.** La primera línea del registro debe decir **Started by an SCM change** en lugar de **Started by user**. El menú lateral del proyecto ofrece además *Git Polling Log*, donde se registra cada consulta al repositorio y si encontró cambios.

Hasta dónde llega el ciclo. Con el disparo automático, la integración continua queda completa: cambio, detección, compilación, prueba y artefacto, sin intervención manual. La entrega continua llega hasta el paquete verificado y disponible. El paso siguiente —desplegar al dispositivo— no se automatiza aquí, y en el dominio embebido rara vez se automatiza: no se detiene una máquina en producción para actualizarla, y un corte durante la escritura puede inutilizar el equipo.

9. Problemas encontrados y su solución

Los siguientes fallos ocurrieron realmente durante la construcción de esta demostración. Se documentan porque es previsible que vuelvan a presentarse.

Síntoma	Causa	Solución
<code>bind source path does not exist: /usr/lib/wsl</code>	BuildKit intenta montar una ruta inexistente al crear su contenedor de construcción; ocurre con Docker Engine en WSL2 sin Docker Desktop	Usar el constructor clásico: <code>DOCKER_BUILDKIT=0 docker build -t imagen:v1 .</code>
Password authentication is not supported	GitHub no acepta contraseñas para operaciones de Git	Generar un token de acceso personal y usarlo como contraseña
Permission denied ... 403 con token válido	El token carece del alcance <code>repo</code> , o es <i>fine-grained</i> sin permiso de escritura	Regenerar el token marcando <code>repo</code> , o conceder <i>Contents: Read and write</i>

Síntoma	Causa	Solución
[rejected] main ->main (fetch first)	El repositorio remoto contiene commits ausentes en local, típicamente el README creado al inicializar	<code>git pull --rebase origin main</code> y volver a publicar
stray '\\' in program	<code>printf</code> con comillas simples no interpreta secuencias de escape, de modo que quedan barras duplicadas	Corregir con <code>sed</code> , o preferir un editor de texto para archivos con macros
missing separator en make	Las reglas del Makefile requieren tabuladores reales, no espacios	<code>sed -i 's/^\t/\t/' Makefile</code> ; verificar con <code>cat -A</code>
Selected Git installation does not exist	Jenkins no tiene registrada la ruta del ejecutable de Git	<i>Administrar Jenkins</i> → <i>Tools</i> → <i>Git installations</i> , ruta <code>/usr/bin/git</code>
Need to specify how to reconcile divergent branches	Se editaron archivos desde la interfaz web mientras se trabajaba en local, generando historias divergentes	<code>git config pull.rebase true</code> y volver a integrar
El disparador no lanza ninguna ejecución	Jenkins aún no ha leído la directiva <code>triggers</code>	Ejecutar una construcción manual para que el disparador quede registrado
El artefacto no aparece en la página de la ejecución	Los artefactos se listan en la página del <i>proyecto</i> , no en la de cada ejecución individual	Consultar <i>Last Successful Artifacts</i> en el proyecto

10. Qué queda demostrado

1. **El entorno de construcción es parte del artefacto de infraestructura.** La imagen de Jenkins debe extenderse para incluir el compilador; de lo contrario el pipeline no puede ejecutar `make`.
2. **El proceso de construcción se versiona como código.** El `Jenkinsfile` vive en el repositorio y cada cambio queda trazable, revisable y reversible.
3. **El pipeline orquesta, no compila.** Toda la lógica reside en el `Makefile`, de modo que lo ejecutado en el servidor se reproduce localmente sin Jenkins.
4. **No todo fallo es equivalente.** Un error de compilación y una prueba fallida tienen consecuencias distintas y se distinguen visualmente.
5. **La política de calidad se aplica sin intervención humana.** Un cambio de un solo carácter, sintácticamente válido, impidió que se produjera un artefacto entregable.
6. **El ciclo se cierra sin intervención humana.** El sondeo del repositorio convierte un *commit* en una construcción verificada, sin que nadie pulse ningún botón.